作业2：列表，字典和类

作者：丁平尖

注意事项

1. 尽早开始！如果在安装或导入包时遇到问题，最好提前发现这些问题，而不是在作业截止前一晚。
2. 确保备份您的工作！推荐学习使用 git。
3. 请先将第一个代码框中的学号修改为自己的学号，再完成以下作业，并按顺序运行各个代码框。

```
import hashlib
import os
import platform
import uuid

# 修改为自己的学号
STUDENT_ID = 20190010

CONDA_ENV = os.environ.get("CONDA_DEFAULT_ENV", "base")
print(f"[Conda 环境] {CONDA_ENV}")

MACHINE_FP = hashlib.sha256(
    f"{platform.node()}|{platform.machine()}|{platform.system()}|
{uuid.getnode()}".encode()
).hexdigest()[:16]
RUN_ID = f"{STUDENT_ID}-{MACHINE_FP}"
print(f"[学号]       {STUDENT_ID}")
print(f"[机器指纹] {MACHINE_FP}")
print(f"[RUN_ID]   {RUN_ID}")
```

作业写作和提交说明

未能遵循这些说明将导致失分。请在 Jupyter Notebook 中完成作业，可在 PyCharm 或 VS Code 中编辑。提交文件名为 `学号_姓名_hw02.ipynb`。

字符串的乐趣

在这个问题中，您将实现一些用于处理字符串的简单函数。您不需要为这个问题中的任何函数执行错误检查。

1. 回文是指一个词或短语，它从前往后读和从后往前读都一样。例如，`level`、`kayak` 和 `pop` 都是英文回文；如果忽略空格和标点符号，`rats live on no evil star` 和 `Was it a car or a cat I saw?` 也是回文短语。编写一个名为 `is_palindrome` 的函数，它接受一个字符串作为唯一参数，并返回布尔值：如果输入是回文，返回 `True`，否则返回 `False`。您可以假设输入仅包含英文字母（不区分大小写）、数字 0 到 9 和空格。判断回文时应忽略空格和大小写，因此 `tacocat` 和 `T A C O cat` 都应被认为是回文。
2. 如果一个词的字母从左到右按字母顺序排列，就称它是“字母顺序的”；允许字母重复。例如，`adder` 和 `beet` 是字母顺序的，`dog` 和 `cat` 则不是。编写一个名为 `is_abecedarian` 的函数，它只接受一个字符串参数。如果参数符合上述要求，返回 `True`，否则返回 `False`。您可以假设输入仅由字母和空格组成。判断时应忽略空格，因此 `abcd efgh xyz` 被认为是字母顺序的。
3. 编写一个名为 `remove_vowels` 的函数，它接受一个字符串作为唯一参数，并返回去掉所有元音后的字符串。本题的元音是字母 `a`、`e`、`i`、`o`、`u`。例如，`remove_vowels('cat')` 应返回 `'ct'`，`remove_vowels('audio')` 应返回 `'d'`。函数也应正确处理全部由元音组成的字符串。

列表的乐趣

在这个问题中，您将实现一些简单的列表操作。

1. 编写一个名为 `list_reverse` 的函数，它接受一个列表作为参数，并返回反转后的列表。例如，输入 `[1, 2, 3]`，应返回 `[3, 2, 1]`。如果输入不是列表，函数应引发适当的错误。
2. 编写一个名为 `is_sorted` 的函数，它接受一个序列 `seq` 作为唯一参数。如果序列按升序排列，返回 `True`，否则返回 `False`。您可以假设 `seq` 是 Python 序列，且所有元素都支持比较操作，因此无需检查输入类型。函数只应遍历序列一次；需要多次遍历的低效解法不能获得满分。这个问题展示了 Python 动态类型的特点：只要 `seq` 支持索引，且元素支持比较，就无需预先知道序列的具体类型。

作为向量的元组

在这个问题中，我们使用元组来表示向量。这里先用一个长度为 d 的元组表示 d 维向量。

1. 实现一个名为 `vec_scalar_mult` 的函数。它接受两个参数：一个由整数或浮点数组成的元组 `t`，以及一个整数或浮点数 `s`。函数返回与 `t` 长度相同的元组，每个元素等于 `t` 的对应元素乘以 `s`。函数应检查输入类型；类型不合适时，引发带有适当错误消息的 `TypeError`。整数标量和含有整数元素的元组都是有效输入。

2. 实现一个名为 `vec_inner_product` 的函数。它接受两个由整数或浮点数组成的元组，返回这两个向量内积对应的浮点数。向量 $x, y \in \mathbb{R}^d$ 的内积是 $\sum_{j=1}^{d} x_j y_j$。若任一输入不是元组，应引发 `TypeError`；若两个元组的长度不一致，应引发 `ValueError`。

3. 将上述表示方法扩展到矩阵。用一个元组的元组表示 $m * n$ 矩阵 `my_mx`：外层元组长度为 m，每个内层元组长度为 n，表示矩阵的一行。编写一个名为 `check_valid_mx` 的函数，它接受一个参数；若该参数是有效矩阵则返回 `True`，否则返回 `False`。有效矩阵满足：

    - 外层元组的每个元素本身都是元组；
    - 所有内层元组长度相同；
    - 每个内层元组的每个元素都是整数或浮点数。

4. 编写一个名为 `mx_vec_mult` 的函数，它接受一个矩阵（元组的元组）和一个向量（元组），返回向量与矩阵相乘得到的数值元组。函数`check_valid_mx`应检查参数是否合理；对于不合理的输入，应引发适当的错误。

更多向量的乐趣

在这个练习中，我们将再次遇到向量，这次采用面向对象的方法。

1. 定义一个名为 `Vector` 的类。每个向量都有一个维度（非负整数）和一个元素列表或元组。初始化函数的第一个参数是维度，第二个参数是由整数或浮点数组成的列表或元组，表示向量的元素。如果只提供维度而不提供元素，请选择合理的默认行为。若维度的类型不正确或为负数，应抛出合适的异常；若维度与元素个数不一致，也应抛出异常。
2. 维度和元素应是类属性还是实例属性？说明选择的理由。
3. 实现向量对象的比较运算符，包括相等、小于、小于或等于、大于等。两个 `Vector` 对象的坐标相同，则视为相等；否则按 Python 元组的字典序比较，即先比较第一个坐标，再比较第二个坐标，依此类推。例如，二维向量 `(2, 4)` 小于 `(2, 5)`。比较维度不同的两个向量时，应抛出异常。
4. 实现 `Vector.dot` 方法，接收另一个 `Vector` 对象，返回两向量的内积。若参数类型不正确或维度不一致，应抛出合适的异常。

5. 实现 `Vector` 的乘法运算符。标量左乘或右乘（如 `2*v` 和 `v*2`）应返回一个各元素都乘以该标量的新 `Vector` 对象。当两个 `Vector` 对象相乘（`v*w`）时，对应位置的元素逐一相乘，结果仍是一个新 `Vector` 对象；例如，`Vector(2, (2, 3)) * Vector(2, (4, 5))` 的元素为 `(8, 15)`。若两个向量的维度不一致，应抛出合适的异常。